Beyond Bounded Unrolling: Certified Loops for Quantum Symbolic Execution

Yanis Djelali, Meng Xu
University of Waterloo, Waterloo, Canada
ydjelali@bordeaux-inp.fr, meng.xu.cs@uwaterloo.ca

***Abstract*—Quantum symbolic execution (QEX) represents program inputs in superposition, but a fixed loop-unrolling depth can omit feasible behaviours. We extend QEX-H's hybrid approach with structure-guided loop analysis and coherent execution. A five-tier classification directs matrix-representable loops to an automatic classical analyzer: it derives transitions from the abstract syntax tree and certifies exact exit analysis through periodicity of matrix powers, including machine wrap-around and non-termination. Reachability engines, invariants and certified unrolling bounds handle the remaining tiers. Quantum circuits evaluate affine and polynomial summaries without repeated body copies, amplify witnesses and estimate violation counts. Independent exhaustive checks validate the classical exit analysis and non-wrapping quantum summaries. On a 20-loop corpus, exit-value under-approximation falls to zero and all 189 routed verdicts agree with the reference. A circuit using the certified depth of 39 reaches a witness missed at 38. IBM hardware measurements quantify the noise cost of repeated loop execution. Together, the results establish a hybrid route from certified loop coverage to coherent state processing.**

***Index Terms*—Program Analysis, Symbolic Execution, Loop Summarization, Constrained Horn Clauses, Bounded Model Checking, Fixed-Point Quantum Search, Quantum Counting**

I. Introduction

Program analysis supports bug detection, security audits and compiler optimization. Undecidability prevents a universal exact analysis [1]: abstract interpretation [2] may include infeasible behaviours, while bounded exploration may omit feasible ones. Symbolic execution [3], [4] faces path explosion and constraint-solving cost.

Guang et al. introduced `QEX` [5], a quantum design for classical program analysis evaluated on 22 benchmarks, following QCheck [6]. Loops remain a limitation: `QEX` duplicates the body circuit a predefined number of times. At the same depth as Angr [7], it reports the same under-approximation rates (Table I). A state beyond the chosen bound remains absent from the quantum encoding.

Extending `QEX`-H's hybrid principle [5], we choose the analysis from the loop structure. The first three tiers admit matrix representations, from which an automatic classical procedure derives exact exits, including machine wrap-around and non-termination. The remaining tiers use reachability, invariant synthesis and certified exploration. In each case, the objective is to replace a guessed iteration limit with justified coverage.

Consider `i=0; while i<n: i+=2`, with nonnegative n and no wrap-around. Unrolling k times misses exits requiring more iterations. The summary $i = 2t$ directly evaluates any candidate count, and $TC = \lceil n/2 \rceil$ gives the first exit. Classical arithmetic handles this loop economically. When a coherent encoding is needed for further analysis, a quantum circuit can evaluate the same summary across superposed inputs and select exits through the guard, without an intermediate measurement. For $n = 5$, it selects $t = 3$ and $i = 6$.

This division of work preserves the purpose of quantum execution: processing encoded states with an oracle that marks a desired property, followed by amplification or counting. We implement affine and polynomial summary circuits, and test certified unrolling when a body is executed step by step. A sufficient bound restores coverage; the circuit's depth and storage requirements determine its cost. Classical constrained Horn clauses (CHC), solved by Spacer [8] or Eldarica [9], Datalog reachability [10], and recurrence diameters [11] provide the analysis foundation.

The contributions are as follows.

- An automatic AST-to-matrix exit analyzer for tiers 1–3, with a periodicity certificate valid under machine wrap-around (Section III-B).
- Coherent affine and polynomial summary circuits, automatic polynomial-body synthesis, and quantum witness search and counting (Section III).
- Classical routing that reuses exact reachable sets and combines complementary invariant engines with certified exploration (Sections IV–V).
- Evaluation on 20 loops at three widths: 189 matching verdicts, zero benchmark under-approximation, a witness recovered at $K = 39$ but missed at 38, complementary classical solver coverage, and measured circuit and hardware costs (Sections VII–VIII).

II. Background and Motivation

A. *QEX in a Nutshell*

`QEX` [5] encodes v inputs of w bits in vw qubits. Quantum Fourier transform (QFT) arithmetic evaluates expressions [12], [13], comparison qubits control branches, and entanglement preserves value dependencies. Measurement samples the encoded relation.

To favour a desired result, an *oracle* marks basis states satisfying a predicate. Write N for the number of candidate basis states and M for the marked ones, so M/N is their fraction under uniform preparation. Our circuits use Grover's

TABLE I
UNDER-APPROXIMATION RATES FOR SYMBOLIC EXECUTION (ANGR) AND QEX WITH BOUNDED LOOP UNROLLING. NUMERICAL DATA REPRODUCED VERBATIM FROM TABLE 3 OF [5].

Test case	Unroll #	SE	QEX
fibonacci	2	33%	33%
divbin2_unwindbound	2	0%	0%
counting	64	71%	71%

$\pi/3$ fixed-point recursion [14]: each level m increases success probability without supplying M/N. This avoids the overshoot of a fixed schedule of ordinary Grover iterations [15]; optimal fixed-point search [16] offers a different query schedule. The recursion level m is distinct from word width w. QEX's coverage metric is

$$\text{under-approximation rate} = \#\text{FN}/\#\text{GT}, \tag{1}$$

where ground truth (GT) contains values arising in actual executions and false negatives (FN) are GT values omitted by the analysis. We retain this metric on exit values.

B. Loops in QEX and the Under-Approximation Problem

Table I isolates the coverage problem. Measuring a guard mid-circuit projects the superposition onto the observed branch; QEX uses bounded body copies instead. States beyond that depth remain missing. We determine how many iterations must be represented, or replace their execution by a summary.

C. Loops as Recurrences and the Halting Limit

Let s_0 be an initial state and g the loop-body update. After t iterations, $s_t = g^t(s_0)$, where the exponent denotes repeated composition. A *closed form* $f(t, s_0)$ evaluates this iterate directly. The trip count TC is the first t at which the guard is false, when the loop terminates. We write k for a chosen unrolling depth and K for a certified sufficient bound.

Unbounded-integer termination is undecidable [17], [18]. At fixed width, reachability is decidable, but explicit exploration may visit exponentially many states. Our classification exploits structure where available.

D. Classical State of the Art: Invariants via Horn Clauses

An inductive invariant Inv contains all reachable loop-head states. A CHC safety proof [19] requires *initiation*, $init \Rightarrow Inv$; *preservation*, $Inv(s) \wedge G(s) \Rightarrow Inv(g(s))$; and *safety*, $Inv \Rightarrow \neg bad$. Here $init$ denotes initial states, G the guard and bad the violating states. Spacer in Z3 [8], [20] and Eldarica [9] seek sufficient invariants, which may include unreachable states.

Datalog's least fixed point under an exact finite-domain encoding is the reachable set itself [10], supporting exact membership and counting. Polar [21] supplies algebraic recurrence solutions. Section V explains their complementary roles.

TABLE II
LOOP CLASSIFICATION AND CLASSICAL ANALYSIS. QUANTUM COMPONENTS ARE DESCRIBED IN SECTIONS III-C–III-F.

Tier	Body structure	Classical analysis
1	constant increments	matrix bound; counter arithmetic
2	coupled linear updates	matrix bound; exact exit search
3	accumulation chains	matrix bound; polynomial shortcuts
4	non-linear updates	Datalog, CHC, certified exploration
5	conditional branches	Datalog, CHC, certified exploration

III. DESIGN

A. Hybrid Strategy and Tier Classification

We adopt QEX-H's principle of combining classical analysis with quantum execution [5]. The front end parses the loop into an *abstract syntax tree* (AST): a structured representation of its guard, assignments, operators and branches. It identifies input variables and dependencies, then classifies the body (Table II). Tier 1 adds constants; tier 2 has coupled linear updates; tier 3 contains acyclic accumulation chains with polynomial iterates, such as `x+=i; i+=1`. Their bodies admit affine matrix representations. Variable products and other non-linear operations enter tier 4; a data-dependent branch enters tier 5.

The automatic classical analysis establishes coverage as indicated in Table II. Quantum processing uses two summary templates (affine increments and polynomial accumulation), a polynomial-body compiler, and witness-search and counting modules. The summary circuits compute arithmetic directly on superposed inputs. Section VI specifies the evaluation of these components.

B. Tiers 1–3: Exact Classical Analysis from the AST

The extractor turns each assignment into a row of B on the augmented state $[s, 1]$; the constant coordinate represents offsets. Thus one body step is $B[s, 1]$ and t steps give $B^t[s_0, 1]$, modulo 2^w, with simultaneous updates. This construction derives the transition, not the trip count.

A bound for every input. There are finitely many matrices over this finite ring, so the powers of B eventually repeat. The analyzer algebraically computes a prefix length $\mu \geq 0$ and repetition period $\lambda \geq 1$, then checks

$$B^{\mu+\lambda} = B^{\mu} \pmod{2^w}, \qquad K = \mu + \lambda - 1. \tag{2}$$

Multiplying the equality by further powers proves periodicity from μ onward. Every state of every input's body orbit therefore occurs at a time $0 \leq t \leq K$. If the guard has not become false by then, it never will. The bound includes the initial state at $t = 0$ and applies to every input, without SMT calls or input enumeration.

Exact exits. For a requested input, test the guard on successive states through K and return the first exit, including $t = 0$. Otherwise report non-termination, keeping it distinct from a zero-valued exit. The result is the relation input $\mapsto$ (terminates, TC, s_{exit}), with TC and the exit defined only for terminating inputs. Arithmetic shortcuts accelerate individual queries; materializing all exits remains a separate cost.

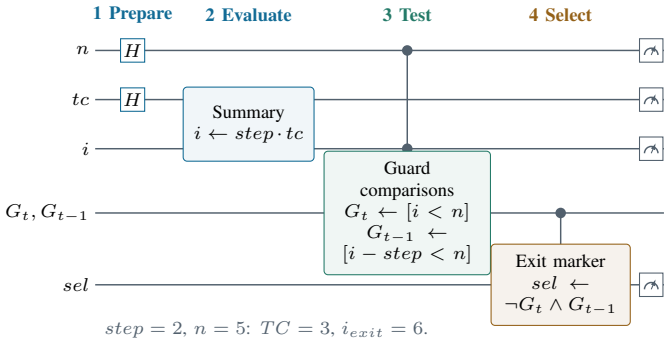


Fig. 1. Coherent affine summary, shown at register level (work qubits omitted). Hadamards H prepare inputs and candidate counts; both guard tests feed the exit marker. Final measurements conditioned on $sel = 1$ sample input–exit pairs. Section III-C states the single-exit conditions.

C. Coherent First-Exit Selection

Figure 1 evaluates the affine summary in four stages. *Prepare*: Hadamards put inputs and candidate counts in uniform superposition. *Evaluate*: arithmetic computes $i = step \cdot tc$. *Test*: signed comparisons encode the current guard $G_t = [i < n]$ and preceding guard $G_{t-1} = [i - step < n]$. *Select*: $\neg G_t \wedge G_{t-1}$ marks the transition to an exit. Postselection samples input–exit pairs, and amplification can favour pairs satisfying a further property. The count $\lceil n/step \rceil$ also follows classically; this circuit demonstrates coherent selection across superposed inputs without intermediate measurement.

Validity. The guard must switch once over the encoded range. Starting at $i = 0$, the evaluator holds $step(2^{|tc|} - 1)$ without overflow, and the count range covers $\lceil (2^w - 1)/step \rceil$. Signed evaluation makes the virtual G_{-1} true, selecting $TC = 0$ when $n = 0$. These conditions derive the range from input width. Agreement with a modular loop additionally requires no wrap-around through its first exit: with step 3 and $n = 7$, a modulo-8 loop takes five iterations, versus three without wrapping. If the guard can switch repeatedly, the first-exit test must also require $G(f(j, s_0))$ for every $j < t$.

D. Quantum Summary Templates

Two parameterized circuit templates implement coherent summaries: affine increments (steps 1, 2 and 3; Fig. 1) and polynomial accumulation. The latter recognizes `while x<y: x += c*i; i += 1`. Direct summation gives

$$i_t = i_0 + t, \qquad x_t = x_0 + c\left(ti_0 + \frac{t(t-1)}{2}\right). \tag{3}$$

We synthesize this arithmetic for $c = 1, 2$, evaluate $x_t < y$, and select the first exit by the construction of Section III-C. For example, $x_0 = 0$, $i_0 = 1$, $c = 1$ gives $x_t = 0, 1, 3, 6$ at $t = 0, 1, 2, 3$; the guard $x_t < 5$ selects $TC = 3$. Inputs already outside the guard select $t = 0$.

For nonnegative inputs and $c \geq 1$, x_t is nondecreasing. Widened arithmetic and a sufficient count range therefore let the selector identify the unique exit. Each circuit computes the summary on superposed inputs without an input–output lookup table. The implemented arithmetic is non-wrapping; Section VII-B reports full-domain tests and identifies the scenarios shared with the modular corpus. General matrix-powering synthesis is discussed in Section VIII.

E. Tier 4: Non-Linear Bodies

Automatic synthesis of the body. Polynomial algebra expands each right-hand side into integer-coefficient monomials ($x \cdot x \mapsto (x, x)$). Supported operations are $+, -, \times$ and constant powers; variable powers, `//` and `%` are explicitly rejected. Each degree-d monomial maps to a QFT phase addition controlled by d registers: degree 0 gives a constant, degree 1 a controlled adder, and degree 2 a multiplier. Register width implements arithmetic modulo 2^w at no additional cost. Verification compares three independently evaluated forms on all 2^{vw} inputs: direct Python interpretation of the AST, its monomial expansion, and the circuit statevector.

Invariant, inductivity and counting. A candidate invariant P can come from Spacer solving generated CHCs or from an external proposal. A Python certifier checks initiation, preservation and safety on its truth table, with corrupted tables as negative controls. The quantum oracle combines that predicate with the synthesized body to mark preservation violations,

$$viol(s) = P(s) \wedge G(s) \wedge \neg P(g(s)),$$

and fixed-point amplification makes them easier to observe. The resulting set supports both witness extraction and cardinality estimation.

Quantum counting [22] estimates the number M of violations among N candidates through phase estimation of the Grover operator. With $p = 7$ phase qubits and 4000 shots, our sign convention gives

$$\hat{M} = \mathrm{round}\big(N \cos^2(\pi y^*/2^p)\big),$$

where y^* is the most frequent measured phase-register value. The phase width p controls resolution; $\hat{M}$ is an estimate. A materialized table also permits exact classical counting. Externally proposed candidates, including LLM suggestions [23], use the same certification and oracle construction. A single SMT query can check preservation independently of invariant synthesis.

F. Tier 5: Data-Dependent Branches

The Collatz template compiles its branch following `QEX`: an ancilla stores the condition, complementary controls select the branches, and the guard qubit freezes exited states. Certified unrolling then supplies complete state coverage using Section IV's bound. A smaller depth gives a "bounded-k" result. The Collatz body's `x // 2` and parity test are implemented as bit operations outside the polynomial compiler.

IV. Certified Unrolling Bound

Repeated body execution needs a *completeness threshold* [24]. We use the initialized recurrence diameter rd: the longest repetition-free path from an initial state [11]. Every reachable state has such a path, so $K \geq \mathrm{rd}$ ensures coverage.

The classical certifier encodes the AST as bit-vector transitions, with symbolic initial states and one disjunct per branch. If SMT excludes a repetition-free path of length k, then $k-1$ is sufficient.

This applies to all five tiers, but computing rd and unrolling to it can be costly. For tiers 1–3, Eq. (2) certifies coverage algebraically, and summary arithmetic can replace repeated bodies. Generic exploration serves loops without these shortcuts and queries requiring a trajectory.

The required bound depends on the query. For existence of a reachable bad state over *all* inputs, breadth-first discovery needs at most $|reach| - |init|$ rounds, each adding a new state. Its fallback is $|D| - |init|$, where D is the finite configuration domain. For *per-input* trajectories and exits, we use rd with symbolic initial states, falling back to $|D| - 1$. A longer deterministic trajectory repeats a state and cannot subsequently exit; state coverage does not imply termination.

For one fixed initial state, the certified path is its trajectory. Iteration controls $[t \geq j]$, where j indexes body copies, let the coherent circuit freeze the final state without storing a guard at each step. Superposed inputs require branch-specific guards.

The K-certifier tries an initial-state witness, exact tier 1–3 evaluation, SMT computation of rd, then the query-appropriate domain bound. Classical witness search advances one iteration at a time and checks witnesses by replay. Coverage comes from the bound; exhaustive exploration, proof or sampling determines the verdict's guarantee.

V. Routing and Reusing Classical Results

Figure 2 organizes the classical routes by the information they produce. Initial-state checks and explicit tier 1–3 evaluation precede Datalog and the CHC portfolio. Exact tables support reusable state queries, invariants support property-directed proofs, and certified exploration supplies a fallback. The implemented stage 1 uses an exact interpreter; the matrix exit analyzer of Section III-B is evaluated separately.

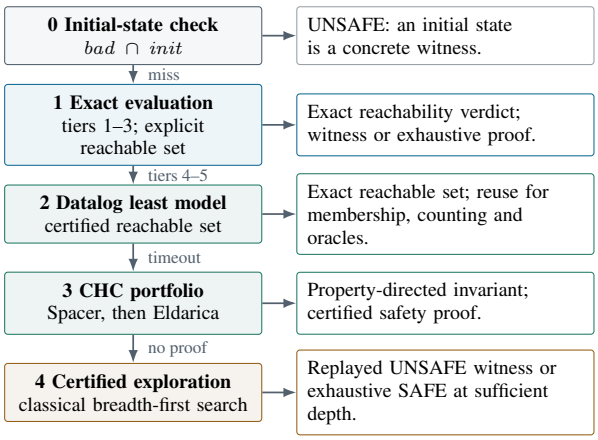


Fig. 2. Classical routing: analysis stages (left) and certified results (right). Datalog materialization precedes property-directed CHC synthesis; the first certified result is retained. Quantum search and counting are evaluated separately using certified bounds or state predicates.

Why Datalog first? For a fixed loop and initial-state set, define $R_0 = init$ and

$$R_{j+1} = R_j \cup \{g(s) : s \in R_j,\ G(s)\}.$$

With the exact finite-domain transition encoding, completed Datalog materialization yields the least fixed point $R_\infty = reach$ [10]. Every inductive invariant containing $init$ contains each R_j, hence $reach \subseteq Inv$: this is the *tightest* invariant for that transition system and initial set. No safety property is needed to construct it. The table can be reused when the property is chosen later: a predicate Q defines the marked set $\{s \in reach : Q(s)\}$. Compiling its membership test into an oracle enables quantum amplification or counting of the selected states. The same materialized table also supports exact classical counting; its construction is shared across queries.

When to use CHC synthesis. If materialization exceeds the budget, Spacer or Eldarica seeks an invariant proving the requested safety property. Its reachable-state over-approximation need not be tight. The distinction is the requested result, not a restriction of CHC syntax: initiation and preservation alone admit even $Inv = D$, the whole domain. Adding a safety clause guides the search but does not request the least model. A loose invariant may prove safety while overcounting reachable states, which is why the router uses certified Datalog tables for exact counting and CHC invariants for proofs.

VI. Methodology

Corpus. We use 20 `while` loops: three tier-1 bodies (`i += 1,2,3`), two tier-3 bodies (`x += i`, `x += 2*i`), a third-order linear recurrence (L06) in tier 2, thirteen tier-4 bodies (products, squares, cubes, `**`, `%`, `//`, coupled updates), and Collatz in tier 5. Branch conditions and all right-hand sides read the pre-iteration state; updates are simultaneous and modulo 2^w. Thus L06 is $(a, b, u) \leftarrow (b, u, a+b)$, not sequential Fibonacci; the conventions coincide on the other 19 loops. Each loop has a free-input scenario, and 19 also have a fixed-precondition scenario. Each receives a violated safety property and, when the reachable set is a strict subset of the domain, a satisfied one, generated from exact reachability: 39 scenarios and 24 satisfied properties give 63 instances per width, at 3, 4 and 5 bits.

Tools. Exhaustive circuit checks use Qiskit [25] and Aer statevector simulation. Classical tools are Z3 4.16 (Spacer and μZ Datalog), Eldarica 2.3 and Polar. Each solver runs separately under a uniform 30 s budget chosen for this study, one solver at a time with virtual-best-solver (VBS) results calculated afterwards, as in CHC-COMP [26].

Evaluation protocol. Following `QEX`'s small-width protocol [5], we compare against exhaustive ground truth and extend the classical corpus evaluation to 4 and 5 bits. Reported circuits execute gates on simulators, except the IBM hardware run of Fig. 4. Predicate and Datalog oracles are truth tables compiled into gates; summary and supported body arithmetic are synthesized directly. Classical routing and reference generation run separately. The affine and polynomial summary

TABLE III

EXIT-VALUE UNDER-APPROXIMATION (1) AT 3 BITS. "OURS" IS COMPUTED CLASSICALLY; POLYNOMIAL CIRCUITS INDEPENDENTLY REPRODUCE THE TWO TIER-3 ROWS. THE GUARD BOUND IS FREE; OTHER INPUTS FOLLOW INIT. GT DENOTES TARGET EXIT VALUES, K^* THE MAXIMAL EXIT TIME AND rd THE CERTIFIED RECURRENCE DIAMETER. SIX TIER-4 ROWS WITH $K^* \leq 2$, ALL AT 0% FOR $k \geq 2$, ARE OMITTED.

Tier	Loop body	Init	$\lvert GT\rvert$	k=1	k=2	k=4	k=8	K^*/rd	ours
T1	`i += 1`	`i=1`	7	71%	57%	29%	0%	6/6	0%
T1	`i += 2`	`i=1`	4	50%	25%	0%	0%	3/3	0%
T1	`i += 3`	`i=1`	3	33%	0%	0%	0%	2/2	0%
T3	`x += i ; i += 1`	`i=1, x=1`	4	50%	25%	0%	0%	3/3	0%
T3	`x += 2 * i ; i += 1`	`i=1, x=1`	3	33%	0%	0%	0%	2/2	0%
T2	`a,b,u := b,u,a+b`	`b=1`	6	83%	83%	83%	33%	10/10	0%
T4	`x = x * i ; i += 1`	`i=1, x=1`	3	67%	33%	0%	0%	3/11	0%
T5	`if x % 2 == 0: ; x = x // 2 ; else: ; x = 3 * x + 1`	`x=7`	1	100%	100%	0%	0%	4/4	0%
T4	`x = x * i + 3 ; i += 1`	`i=1, x=1`	3	33%	33%	33%	0%	6/6	0%
T4	`x = (x + 1) * (i + 2) ; i += 1`	`i=1, x=1`	3	33%	33%	33%	33%	9/9	0%
T4	`x = x * x + i ; i += 1`	`i=1, x=1`	4	50%	25%	0%	0%	3/3	0%
T4	`x = x * i - i ; i += 1`	`i=1, x=1`	3	67%	33%	0%	0%	3/3	0%
T4	`x = x % 3 + i ; i += 1`	`i=1, x=1`	5	60%	40%	20%	0%	5/5	0%
T4	`x = x // 2 + i ; i += 1`	`i=1, x=1`	4	75%	50%	0%	0%	4/7	0%

experiments use widened, non-wrapping arithmetic, with their relationship to the modular corpus stated explicitly.

Verification. An independent AST interpreter checks termination, trip count and complete exit state by cycle detection; the same interpreter computes fixed points to check all 189 routed verdicts and reachable-set sizes. Archived solvers corroborate the 42 stage-1 verdicts. A deliberately sequential interpreter detects the L06 semantic difference. Polynomial summaries use exhaustive statevectors and faulty-circuit controls (Section VII-B); AST evaluation checks monomial expansion. The 401-qubit experiment validates a complete deterministic trace on basis states.

Objects and metrics. $reach$ is the set of loop-head states over all inputs of a scenario; *exit* states are those on which the guard is false; the relation $\{(s_0, s_{exit})\}$ links each input to its exit. Properties, as in the CHC encoding, concern loop-head reachability of a bad state; Table III concerns exit values of the target variable. We measure under-approximation (1) on exit values, invariant *tightness* $|Inv|/|reach|$, disagreements with brute force, and transpiled qubit counts, gate counts and depth.

VII. EXPERIMENTAL RESULTS

A. Exact Exit Coverage

Applying Table I's metric to our corpus gives Table III. At k=2, bounded unrolling omits 83% of L06's exit values and 57% for `i += 1`. The rate reaches 0% at $k \geq K^*$, but a guessed bound need not reach this threshold. Exact classical analysis gives 0% throughout: the matrix-certified exit analyzer for tiers 1–3 and exhaustive exploration at K=rd for tiers 4–5. SMT establishes $\mathrm{rd} \geq K^*$ for every loop. Nontermination can make rd exceed K^*, as in `x = x*i` (11 versus 3), because the diameter also covers paths that do not exit.

Classical exit-analysis validation. The analyzer passes 268 cases covering 939 968 inputs, including 10 063 nonterminating ones. The reference agrees on termination, trip count and exit state; explicit power exploration validates certificates, and negative controls are rejected. All 500 sampled 16-bit inputs also agree. For L06 at 5 bits, $K = 111$ takes 5.3 ms to certify, versus $\mathrm{rd} = 93$ in 0.63 s by SMT. At 32 bits, $K = 15\,032\,385\,535$ is certified in 16.7 ms; this measures bound construction, not full exit enumeration.

B. Coherent Summary Circuits

The affine circuit (step 2, predicate $i = 6$) selects exactly $n \in \{5, 6\}$. At fixed-point level $m = 4$, repeated 4000-shot runs give success frequencies 0.921–0.923, compared with the ideal probability 0.924. The marked input–exit relation is evaluated directly in gates.

Polynomial summaries. For each of $c = 1, 2$, Eq. (3) is synthesized with a 3-bit iteration register. Scenario S1 ($x_0 = i_0 = 1$, free 3-bit y) validates all 64 input–count branches and rejects three faulty circuits. Without wrap-around before exit, it reproduces the two tier-3 rows of Table III. With all three inputs free, each circuit validates all 512 inputs and 4096 input–count branches; a faulty-circuit control is rejected for each. This larger experiment evaluates non-wrapping integer loops: modulo-8 execution differs on 88 and 156 inputs, respectively. Table VII reports the synthesized resources.

TABLE IV

IDENTICAL UNROLLING CIRCUIT WITH CERTIFIED AND GUESSED k. THE TIER-4 LOOP `WHILE X<Y: X=X*I; I+=1` HAS A NON-INJECTIVE BODY; INPUTS ARE FIXED TO x=i=1, y=31 AT 5 BITS. ITS REACHABLE SET HAS $|reach| = 40$ AND EXACT BFS DEPTH 39.

	ours: certified K	QEX-style: guessed k
k used	$39 = \mathrm{rd}$	e.g. 38
status of k	certified: SMT unsat at 40; rd equals BFS depth; 0.1 s	none (arbitrary bound)
false property: state (i, x)=$(8, 0)$, reached only at t=39	UNSAFE; witness measured at t=39, replayed	nothing found at 38: "SAFE bounded-38"; a global SAFE would be unjustified
true property: x=3 unreachable	SAFE, exhaustive (deterministic read-out, trace = $reach$)	SAFE "bounded-k", no guarantee beyond k
under-approx.	eliminated ($K \geq \mathrm{rd}$)	present by construction

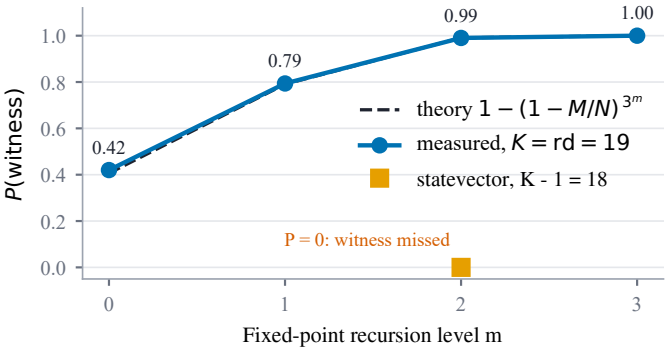


Fig. 3. Coherent witness search for `while x<y: x+=i; i+=1`, starting at $x = i = 1$, $y = 31$ (5 bits, 16 qubits). The free iteration register has $N = 32$ time labels; $M = 13$ labels ($t = 19, \ldots, 31$) encode the same frozen exit $(i, x) = (20, 31)$. Level m counts fixed-point recursions. At $K = \mathrm{rd} = 19$, plotted frequencies use 4000 shots; statevector probabilities match theory within 10^{-6}. All 13 measured time-labelled witnesses replay. The $K-1 = 18$ baseline denotes absence, checked by statevector at $m = 2$. The circuit receives no value of M/N.

C. Certified K in Circuit: K versus $K-1$

In Table IV, $(i, x) = (8, 0)$ first appears at $t = 39$ in a 40-state trajectory. From $t = 8$, x stays zero and the guard remains true: this is loop-head reachability in a non-terminating execution. SMT certifies $\mathrm{rd} = 39$ in 0.1 s; BFS independently checks that value. Non-injective updates retain new state registers [27], giving 401 qubits and depth 2.1×10^5. MPS executes the circuit on basis states in 9 s. Its complete measured trace equals the reference reachable set, proving the safe property exhaustively and supplying a replayed witness for the false one. At 38, the same circuit misses that witness.

Figure 3 exercises the same certified-unrolling principle coherently on a tier-3 loop. Its free iteration register allows witness amplification, following $1-(1-M/N)^{3^m}$ without supplying M/N. This experiment isolates the effect of a sufficient bound independently of summary evaluation. The safe-property and $K-1$ checks use the statevector at $m = 2$: every witness-labelled probability is below 10^{-9}, consistent with the exact classical absence checks.

Tier 5 at the certified depth. SMT supplies $K = \mathrm{rd} = 4$ for all 3-bit Collatz inputs directly to the circuit constructor. Exhaustive statevectors validate all eight trajectories of the 17-qubit circuit (depth 165) against an independent reference; 4000-shot measurements, visited states and trip counts agree per input. At $K-1 = 3$, the same construction misses $x = 1$ from $x_0 = 7$; a $3x+2$ mutant is rejected on four inputs.

D. Classical Invariant Engines by Width

The classical results in Table V motivate the router. Every returned verdict matches brute force and the other engines. At 5 bits, Spacer and Datalog time out on eleven scenarios with three free input variables ($|reach| = 32\,768$); Eldarica resolves these in roughly 0.9 s. Conversely, eleven bounded non-linear instances time out in Eldarica but take Spacer 0.1 s. Their disjoint failures let the virtual best solver decide all 63. Eldarica returns loose invariants that sometimes differ across runs: fixed L06 at 3 bits gives 12 versus 87 states. For free L06 at 5 bits, all five runs give a certified $|Inv| = 1\,015\,808$ against $|reach| = 6\,323$, inflating counts by $\times 160$. Materializing the 20 Datalog tables takes a total of 0.11 s, 0.44 s and 5.6 s at 3, 4 and 5 bits. The eleven 5-bit Spacer/Datalog timeouts are degenerate cases: every variable is an input, $init = reach$, and router stage 0 decides each property at $K{=}0$.

TABLE V
CLASSICAL INVARIANT ENGINES ON 63 INSTANCES WITH 30 S BUDGETS. DECIDED COUNTS SAT/UNSAT ANSWERS. TIGHT COUNTS INVARIANTS EQUAL TO THE REACHABLE SET, ONCE PER LOOP; TIMEOUTS ARE NOT TIGHT. VBS COMBINES SPACER AND ELDARICA. POLAR CERTIFIES 0 INSTANCES: 18 REFUSALS AND 2 EXACT POLYNOMIAL IDENTITIES VALID MODULO 2^w BUT INSUFFICIENT FOR SAFETY.

	Decided / 63			Tight / 20		
	3 b	4 b	5 b	3 b	4 b	5 b
Spacer (PDR)	63	63	52	20	20	20
Eldarica (pred. abs.)	63	52	52	10	9	9
Datalog (μZ)	63	63	52	20	20	20
VBS Spacer+Eldarica	63	63	63	–	–	–

E. Routed Runs and Quantum Counting

Across 63 instances at each width, routing has the same distribution: stage 0 resolves 16, stage 1 resolves 14, and stage 2 resolves 33; stages 3–4 are unused. All 189 verdicts agree with the independent interpreter, and all 33 Datalog tables per width equal their reachable sets. Disabling earlier stages exercises the later classical routes: 59 forced 3-bit runs produce no contradictions. Stage 4 finds 13 replayed UNSAFE witnesses and proves 14 SAFE verdicts by exhaustive exploration to a fixed point. The certified rd equals BFS depth in 13 of these SAFE cases; the other has rd=1 for depth 0, a sufficient bound.

The quantum experiments use these exact relations as references. Counting on a certified Datalog table for `x = x*i` gives $\hat{M} = 12$ for $M = 12$ marked states among $N = 64$ ($p = 7$, 4000 shots). The inductivity circuit tests hand-written candidates with synthesized body arithmetic. "x even" gives $\hat{M} = 0$ and "$x \neq 0$" gives $\hat{M} = 11$, matching the classical violation counts. For the latter, measured amplification frequencies are $0.177, 0.419, 0.818, 0.995$ at $m = 0, \ldots, 3$, versus predicted $0.172, 0.432, 0.817, 0.994$. Counting remains an estimate; the classical checks supply the corresponding proofs and exact counts.

F. Resource Consumption

The synthesized summaries in Table VII use up to 19 qubits for tier 1 and 26 for tier 3. Their arithmetic depth depends on register width, not the numerical trip count; amplification adds the cost of selecting marked branches. Non-injective unrolling instead retains roughly one garbage register per iteration, reaching 401 qubits at $K{=}39$.

VIII. RESOURCE TRADE-OFFS AND SCOPE

Table VI distinguishes coverage from execution cost: a fast certificate can authorize a large unrolling.

TABLE VI

COVERAGE AND RESOURCE TRADE-OFFS. CERTIFICATES ESTABLISH WHAT MUST BE REPRESENTED; CIRCUIT COSTS DETERMINE HOW TO REPRESENT IT.

Aspect	Where	Evidence	Design implication
Reversibility (Bennett)	non-injective unrolled bodies	401 qubits at K=39, 5 bits; a frozen guard costs one ancilla per step even for injective bodies	deep coherent non-injective unrolling can exceed simulation and hardware budgets; certified K exposes this cost
Fixed-point cost	recursion level m	$\pi/3$ recursion has cost $O(3^m)$	monotone amplification follows the predicted probability law; optimal fixed-point scheduling [16] offers lower query cost
Hardware noise	unrolled increment	Fig. 4: fidelity $0.99 \to 0.38$ across 45 iterations (402 two-qubit gates), above the $1/8$ floor	measured baseline for repeated body execution; richer bodies require more gates and qubits
Finite-state scaling	modular state graphs	matrix certification can be fast even when K is large (L06, 32 bits: $K > 15$ billion)	a certificate removes an arbitrary cutoff; full relation materialization and unrolling can remain expensive
Counting complexity	oracle queries	classical formulas count simple predicates; materialized exact tables also permit classical counting	quantum estimation offers established query gains [22], [28]; unstructured search is capped at quadratic [29]. Oracle construction is excluded from that comparison; error-correction costs matter [30]

TABLE VII

SYNTHESIZED CIRCUIT RESOURCES AT 3-BIT INPUTS UNLESS NOTED. PAIRED TIER-3 ENTRIES CORRESPOND TO $c = 1, 2$; S1 FIXES $x_0 = i_0 = 1$, WHILE "FULL" LEAVES x_0, i_0, y FREE. SUMMARY DEPTHS EXCLUDE AMPLIFICATION; "PREP." DENOTES PREPARATION.

Circuit	Level	Backend	Qubits	Depth / gates
T1 four-step, step 2, 3	gates	Aer	19, 18	depth 233, 248
T3 summaries, S1	gates	statevec.	17, 18	depth 465, 540
T3 summaries, full	gates	statevec.	25, 26	depth 955, 1075
T4 body g, polynomial	gates	statevec.	≤ 12	exact on 64/64
T5 Collatz, K=4	gates	statevec.	17	depth 165
K=19 coherent hunt, 5 b	gates	Aer	16	2 321 gates (prep.)
K=39 trajectory, 5 b	gates	MPS	401	depth 2.1×10^5
`x+=1` ×45, QPU	gates	marrakesh	3	402 2q gates

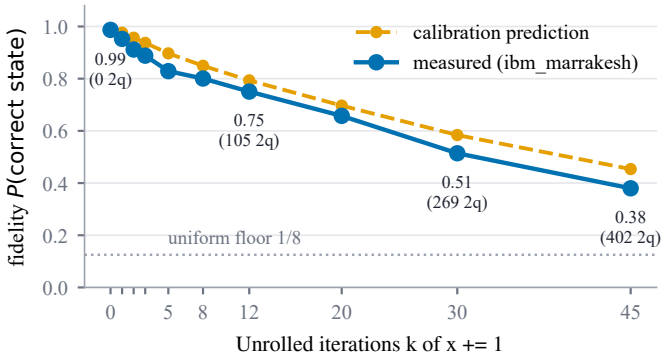


Fig. 4. State fidelity on ibm_marrakesh after k iterations of a 3-qubit increment (2026-08-13; 4096 shots; one grouped job; no error mitigation). Barriers prevent the transpiler from merging iterations; about 9 two-qubit gates per iteration. Measurements follow the calibration-based prediction from below and remain above the uniform floor throughout the campaign.

The IBM experiment measures fidelity falling to 0.38 after 45 increments, still three times the uniform floor. This motivates structural summaries before repeated body execution.

Quantum synthesis covers the templates and body operations of Section III. General tier-2 synthesis would apply powers $B, B^2, B^4, \ldots$ controlled by count bits, following Shor [31]. The experiments establish circuit correctness and resource costs on the stated domains; a practical quantum advantage would additionally require economical oracle preparation and fault-tolerant execution.

IX. RELATED WORK

Quantum and classical loop analysis. QEX [5] and QCheck [6] encode classical programs as quantum circuits; QEX-H motivates our hybrid organization. Loop-extended symbolic execution [32] introduces symbolic loop counts, while finite-monoid acceleration [33] exploits periodic affine relations over integers. Our analyzer uses periodic matrix powers over a finite machine-arithmetic ring to certify exact exits, paired with quantum summary and search experiments.

Invariant synthesis and Horn clauses. Classical loop analysis uses Spacer [8], Eldarica [9], μZ [10] and CHC encodings [19]. Polar [21] derives algebraic recurrence solutions; externally proposed invariants [23] share the same certification obligations; exact reachable-state counting additionally requires tightness.

Bounded model checking. We use recurrence diameters [11] as completeness thresholds [24], [34]. An alternative for proving safety is k-induction [35], outside this evaluation.

Quantum counting and estimation. Counting [22] and amplitude estimation [36] provide query-complexity gains used in QPMC [37] and Monte-Carlo estimation [28]. Unstructured search is limited to quadratic improvement [29]; error-correction costs can erase a practical gain [30].

X. CONCLUSION AND FUTURE WORK

We extend QEX-H's hybrid approach with structure-guided, certified loop handling. Automatic matrix analysis computes exact modular exits for tiers 1–3, while reachability and invariants support the remaining tiers. The evaluated corpus has zero exit-value under-approximation, with independent agreement on 939 968 exit-analysis inputs and all 189 routed verdicts. Quantum summary circuits select exits coherently, certified unrolling recovers a witness missed at a smaller depth, and amplification and counting process the resulting state predicates. Resource measurements, including IBM hardware, explain when summary evaluation is preferable to repeated body execution.

Further work extends matrix-powering synthesis, polynomial templates and supported arithmetic, and evaluates optimal fixed-point scheduling and hardware amplification with certified state oracles.

References

[1] W. Landi, "Undecidability of static analysis," *ACM Letters on Programming Languages and Systems*, vol. 1, no. 4, pp. 323–337, 1992.

[2] P. Cousot and R. Cousot, "Abstract interpretation: A unified lattice model for static analysis of programs by construction or approximation of fixpoints," in *Proceedings of the 4th ACM SIGACT-SIGPLAN Symposium on Principles of Programming Languages (POPL)*, 1977, pp. 238–252.

[3] J. C. King, "Symbolic execution and program testing," *Communications of the ACM*, vol. 19, no. 7, pp. 385–394, 1976.

[4] R. Baldoni, E. Coppa, D. C. D'Elia, C. Demetrescu, and I. Finocchi, "A survey of symbolic execution techniques," *ACM Computing Surveys*, vol. 51, no. 3, pp. 1–39, 2018.

[5] Y. Guang, P. Zanotta, K. Zhou, Y. Chen, and R. Ayanzadeh, "On the potential of quantum computing in classical program analysis," in *2025 IEEE International Conference on Quantum Computing and Engineering (QCE)*. IEEE, 2025, pp. 2364–2374.

[6] ——, "Quantum methods for boundary checking in classical programs," in *Proceedings of the 23rd Annual International Conference on Mobile Systems, Applications and Services (MobiSys)*, 2025, pp. 775–778.

[7] Y. Shoshitaishvili, R. Wang, C. Salls, N. Stephens, M. Polino, A. Dutcher, J. Grosen, S. Feng, C. Hauser, C. Kruegel, and G. Vigna, "SoK: (state of) the art of war: Offensive techniques in binary analysis," in *2016 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2016, pp. 138–157.

[8] A. Komuravelli, A. Gurfinkel, and S. Chaki, "SMT-based model checking for recursive programs," *Formal Methods in System Design*, vol. 48, no. 3, pp. 175–205, 2016.

[9] H. Hojjat and P. Rümmer, "The ELDARICA Horn solver," in *2018 Formal Methods in Computer Aided Design (FMCAD)*. IEEE, 2018, pp. 1–7.

[10] K. Hoder, N. Bjørner, and L. de Moura, "μZ – an efficient engine for fixed points with constraints," in *Computer Aided Verification (CAV 2011)*, ser. Lecture Notes in Computer Science, vol. 6806. Springer, 2011, pp. 457–462.

[11] D. Kroening and O. Strichman, "Efficient computation of recurrence diameters," in *Verification, Model Checking, and Abstract Interpretation (VMCAI 2003)*, ser. Lecture Notes in Computer Science, vol. 2575. Springer, 2003, pp. 298–309.

[12] T. G. Draper, "Addition on a quantum computer," arXiv:quant-ph/0008033, 2000.

[13] L. Ruiz-Perez and J. C. Garcia-Escartin, "Quantum arithmetic with the quantum Fourier transform," *Quantum Information Processing*, vol. 16, p. 152, 2017.

[14] L. K. Grover, "Fixed-point quantum search," *Physical Review Letters*, vol. 95, p. 150501, 2005.

[15] ——, "A fast quantum mechanical algorithm for database search," in *Proceedings of the 28th Annual ACM Symposium on Theory of Computing (STOC)*, 1996, pp. 212–219.

[16] T. J. Yoder, G. H. Low, and I. L. Chuang, "Fixed-point quantum search with an optimal number of queries," *Physical Review Letters*, vol. 113, p. 210501, 2014.

[17] A. M. Turing, "On computable numbers, with an application to the Entscheidungsproblem," *Proceedings of the London Mathematical Society*, vol. s2-42, no. 1, pp. 230–265, 1937.

[18] H. G. Rice, "Classes of recursively enumerable sets and their decision problems," *Transactions of the American Mathematical Society*, vol. 74, no. 2, pp. 358–366, 1953.

[19] N. Bjørner, A. Gurfinkel, K. McMillan, and A. Rybalchenko, "Horn clause solvers for program verification," in *Fields of Logic and Computation II*, ser. Lecture Notes in Computer Science, L. D. Beklemishev, A. Blass, N. Dershowitz, B. Finkbeiner, and W. Schulte, Eds. Springer, 2015, vol. 9300, pp. 24–51.

[20] L. de Moura and N. Bjørner, "Z3: An efficient SMT solver," in *Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*, ser. LNCS, vol. 4963. Springer, 2008, pp. 337–340.

[21] M. Moosbrugger, M. Stankovič, E. Bartocci, and L. Kovács, "This is the moment for probabilistic loops," *Proceedings of the ACM on Programming Languages*, vol. 6, no. OOPSLA2, pp. 1497–1525, 2022.

[22] G. Brassard, P. Høyer, and A. Tapp, "Quantum counting," in *Automata, Languages and Programming (ICALP 1998)*, ser. Lecture Notes in Computer Science, vol. 1443. Springer, 1998, pp. 820–831.

[23] V. Bharti, S. Jha, D. Kumar, and P. Jalote, "Loop invariant generation: A hybrid framework of reasoning optimised LLMs and SMT solvers," arXiv:2508.00419, 2025.

[24] E. Clarke, D. Kroening, J. Ouaknine, and O. Strichman, "Completeness and complexity of bounded model checking," in *Verification, Model Checking, and Abstract Interpretation (VMCAI 2004)*, ser. Lecture Notes in Computer Science, vol. 2937. Springer, 2004, pp. 85–96.

[25] A. Javadi-Abhari, M. Treinish, K. Krsulich, C. J. Wood, J. Lishman, J. Gacon, S. Martiel, P. D. Nation, L. S. Bishop, A. W. Cross, B. R. Johnson, and J. M. Gambetta, "Quantum computing with Qiskit," arXiv:2405.08810, 2024.

[26] E. De Angelis and H. G. Vediramana Krishnan, "Competition of solvers for constrained Horn clauses (CHC-COMP 2023)," in *TOOLympics Challenge 2023: Updates, Results, Successes of the Formal-Methods Competitions*, ser. Lecture Notes in Computer Science, D. Beyer, A. Hartmanns, and F. Kordon, Eds. Cham: Springer, 2025, vol. 14550, pp. 38–51.

[27] C. H. Bennett, "Logical reversibility of computation," *IBM Journal of Research and Development*, vol. 17, no. 6, pp. 525–532, 1973.

[28] A. Montanaro, "Quantum speedup of Monte Carlo methods," *Proceedings of the Royal Society A: Mathematical, Physical and Engineering Sciences*, vol. 471, no. 2181, p. 20150301, 2015.

[29] C. H. Bennett, E. Bernstein, G. Brassard, and U. Vazirani, "Strengths and weaknesses of quantum computing," *SIAM Journal on Computing*, vol. 26, no. 5, pp. 1510–1523, 1997.

[30] R. Babbush, J. R. McClean, M. Newman, C. Gidney, S. Boixo, and H. Neven, "Focus beyond quadratic speedups for error-corrected quantum advantage," *PRX Quantum*, vol. 2, p. 010103, 2021.

[31] P. W. Shor, "Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer," *SIAM Journal on Computing*, vol. 26, no. 5, pp. 1484–1509, 1997.

[32] P. Saxena, P. Poosankam, S. McCamant, and D. Song, "Loop-extended symbolic execution on binary programs," in *Proceedings of the International Symposium on Software Testing and Analysis (ISSTA)*. ACM, 2009, pp. 225–236.

[33] M. Bozga, R. Iosif, and F. Konečný, "Fast acceleration of ultimately periodic relations," in *Computer Aided Verification (CAV)*, ser. Lecture Notes in Computer Science, vol. 6174. Springer, 2010, pp. 227–242.

[34] A. Biere, A. Cimatti, E. M. Clarke, and Y. Zhu, "Symbolic model checking without BDDs," in *Tools and Algorithms for the Construction and Analysis of Systems (TACAS 1999)*, ser. Lecture Notes in Computer Science, vol. 1579. Springer, 1999, pp. 193–207.

[35] M. Sheeran, S. Singh, and G. Stålmarck, "Checking safety properties using induction and a SAT-solver," in *Formal Methods in Computer-Aided Design (FMCAD 2000)*, ser. Lecture Notes in Computer Science, vol. 1954. Springer, 2000, pp. 127–144.

[36] G. Brassard, P. Høyer, M. Mosca, and A. Tapp, "Quantum amplitude amplification and estimation," 2000, arXiv:quant-ph/0005055.

[37] S. Jeon, K. Cho, C. G. Kang, J. Lee, H. Oh, and J. Kang, "Quantum probabilistic model checking for time-bounded properties," *Proceedings of the ACM on Programming Languages*, vol. 8, no. OOPSLA2, pp. 557–587, 2024.